

## **MVC PHP TD1**

### **Sommaire**

|                                           |   |
|-------------------------------------------|---|
| <a href="#"><u>Introduction</u></a> ..... | 1 |
| <a href="#"><u>Exercice:1</u></a> .....   | 2 |
| <a href="#"><u>Exercice :2</u></a> .....  | 3 |
| <a href="#"><u>Exercice:3</u></a> .....   | 4 |
| <a href="#"><u>Exercice :4</u></a> .....  | 5 |
| <a href="#"><u>Exercice:5</u></a> .....   | 6 |
| <a href="#"><u>Exercice : 6</u></a> ..... | 6 |
| <a href="#"><u>Exercice:7</u></a> .....   | 6 |
| <a href="#"><u>Exercice : 8</u></a> ..... | 7 |
| <a href="#"><u>Exercice : 9</u></a> ..... | 7 |
| <a href="#"><u>Conclusion</u></a> .....   | 7 |

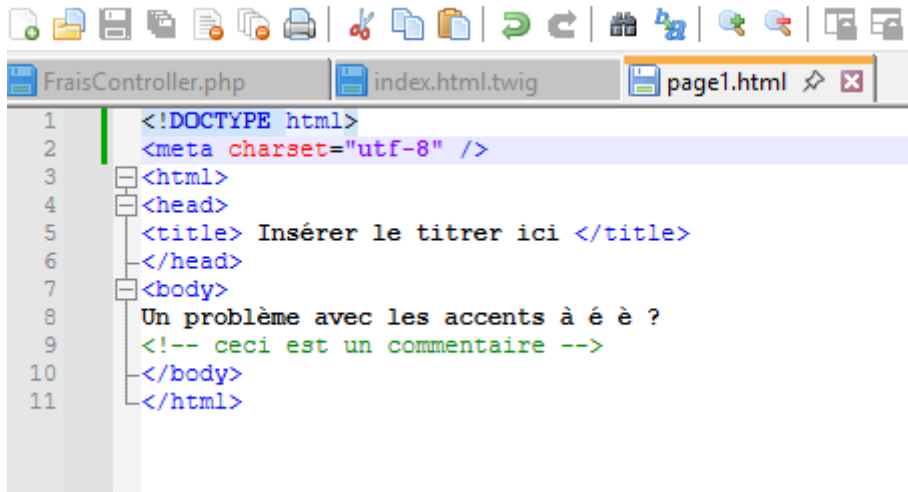
### **Introduction**

Dans ce TP, nous allons découvrir les bases du développement web avec PHP. Nous allons apprendre à créer et afficher des pages HTML, comprendre le fonctionnement d'un serveur web et utiliser progressivement PHP pour rendre nos pages dynamiques.

## **Exercice:1**

Création de page1.html

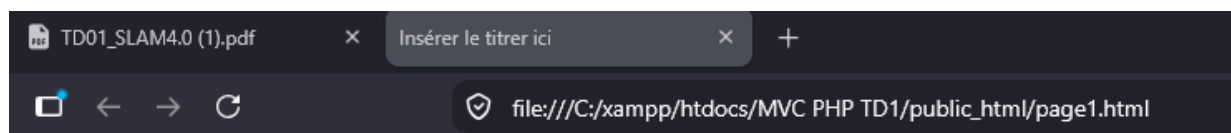
Le fichier page1.html a été placé dans le dossier public\_html.



```
1 <!DOCTYPE html>
2 <meta charset="utf-8" />
3 <html>
4 <head>
5 <title> Insérer le titrer ici </title>
6 </head>
7 <body>
8 Un problème avec les accents à é è ?
9 <!-- ceci est un commentaire -->
10 </body>
11 </html>
```

### **Ouverture en double-cliquant sur le fichier :**

On remarque bien l'url obtenue .



Un problème avec les accents à é è ?

### **Que fait le gestionnaire de fichiers quand on double-clique ?**

Il ouvre le fichier page1.html avec le navigateur web par défaut.

Que signifie file au début de l'URL ?

file:// indique que le navigateur ouvre directement un fichier présent sur l'ordinateur, et non une page provenant d'un serveur web.

### Est-ce que la page HTML s'affiche correctement ?

Oui, la page doit s'afficher dans le navigateur. La ligne :

`<meta charset="utf-8" />`

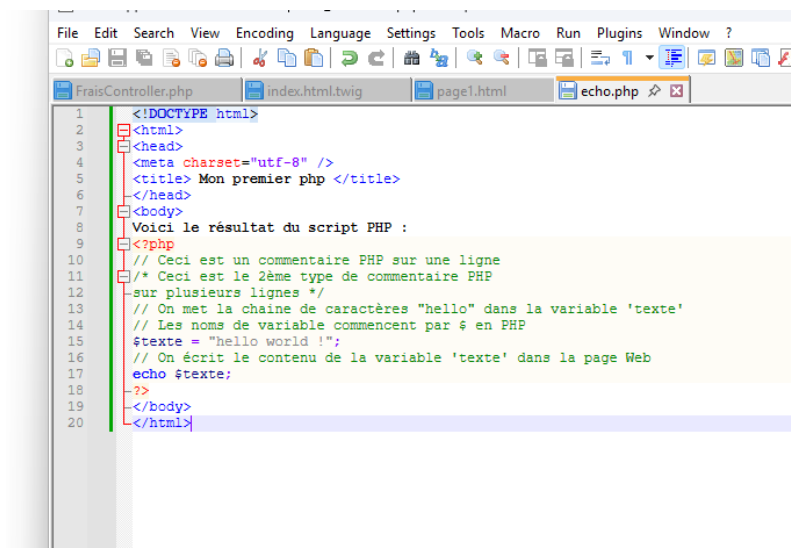
permet notamment de gérer correctement les accents.

### Est-ce qu'il y a une communication entre un serveur et un client HTTP ?

Non. Le fichier est ouvert directement depuis l'ordinateur avec file://. Il n'y a donc pas de requête HTTP entre un client et un serveur.

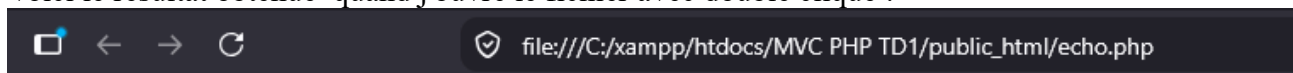
## Exercice :2

Création de echo.php



```
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <meta charset="utf-8" />
5 <title> Mon premier php </title>
6 </head>
7 <body>
8 Voici le résultat du script PHP :
9 <?php
10 // Ceci est un commentaire PHP sur une ligne
11 /* Ceci est le 2ème type de commentaire PHP
12 sur plusieurs lignes */
13 // On met la chaîne de caractères "hello" dans la variable 'texte'
14 // Les noms de variable commencent par $ en PHP
15 $texte = "hello world !";
16 // On écrit le contenu de la variable 'texte' dans la page Web
17 echo $texte;
18 ?>
19 </body>
20 </html>
```

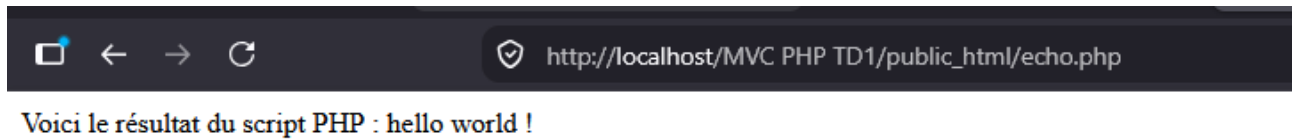
Voici le résultat obtenu quand j'ouvre le fichier avec double clique :



Voici le résultat du script PHP :

ALOUANI Fadi

puis quand je le l'ouvre avec localhost :



## Pourquoi ?

Cela ne fonctionne pas car le fichier est ouvert directement par le navigateur avec `file://`. Alors que le code PHP doit être exécuté par un serveur comme Apache.

Quand j'ouvre la page dans le navigateur en passant par le serveur HTTP je lance Apache avec XAMPP puis j'accède à `echo.php` avec l'adresse `http://localhost/....` La page affiche le résultat du script PHP. En faisant `Ctrl + U`, je peux voir le code HTML généré par PHP, mais pas le code PHP lui-même, car celui-ci a été exécuté par le serveur Apache.

## Exercice:3

J'ai initialisé un dépôt Git dans le dossier `public_html` avec la commande `git init`. J'ai ensuite vérifié l'état du dépôt avec `git status`, ajouté le fichier `echo.php` avec `git add echo.php`, puis enregistré les modifications avec `git commit`. Enfin, j'ai utilisé `git status` pour vérifier que le dépôt était à jour.

```
C:\xampp\htdocs\MVC PHP TD1\public_html>git init
Initialized empty Git repository in C:/xampp/htdocs/MVC PHP TD1/public_html/.git/

C:\xampp\htdocs\MVC PHP TD1\public_html>git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    echo.php
    page1.html

nothing added to commit but untracked files present (use "git add" to track)

C:\xampp\htdocs\MVC PHP TD1\public_html>git add
Nothing specified, nothing added.
hint: Maybe you wanted to say "git add ."
hint: Disable this message with "git config set advice.addEmptyPaths false"

C:\xampp\htdocs\MVC PHP TD1\public_html>git add echo.php

C:\xampp\htdocs\MVC PHP TD1\public_html>git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   echo.php

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    page1.html

C:\xampp\htdocs\MVC PHP TD1\public_html>git commit -m "TD1 Exo2 Premiere page php"
[master (root-commit) c81b88f] TD1 Exo2 Premiere page php
1 file changed, 20 insertions(+)
create mode 100644 echo.php

C:\xampp\htdocs\MVC PHP TD1\public_html>git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    page1.html

nothing added to commit but untracked files present (use "git add" to track)

C:\xampp\htdocs\MVC PHP TD1\public_html>
```

## Exercice :4

Dans cet exercice, j'ai créé trois variables contenant la marque, la couleur et l'immatriculation d'une voiture. Je les ai ensuite affichées avec la commande echo. J'ai également créé un tableau associatif contenant les informations d'une voiture et utilisé var\_dump pour vérifier son contenu. Enfin, j'ai créé une liste de plusieurs voitures et utilisé une boucle foreach pour afficher leurs informations dans une liste HTML. La fonction empty() permet de vérifier si la liste est vide.

```

1 <!DOCTYPE html>
2 <html>
3 <head>
4 <meta charset="utf-8" />
5 <title>Mon premier PHP</title>
6 </head>
7 <body>
8
9 Voici le résultat du script PHP :
10
11 <?php
12
13 // 1. Création des variables
14 $marque = "Renault";
15 $couleur = "bleu";
16 $immatriculation = "256AB34";
17
18 // Affichage des variables
19 echo "<p>Voiture " . $immatriculation . " de marque " . $marque . " (couleur " . $couleur . " )";
20
21
22 // 2. Création d'un tableau associatif
23 $voiture = array(
24     "marque" => "Peugeot",
25     "couleur" => "rouge",
26     "immatriculation" => "123CD45"
27 );
28
29 // Vérification du contenu du tableau
30 var_dump($voiture);
31
32 // Affichage du tableau
33 echo "<p>Voiture " . $voiture["immatriculation"] .
34     " de marque " . $voiture["marque"] .
35     " (couleur " . $voiture["couleur"] . " )</p>";
36
37
38 // 3. Création d'une liste de voitures
39 $voitures = array(
40     array(
41         "marque" => "Renault",
42         "couleur" => "bleu",
43         "immatriculation" => "256AB34"
44     ),
45     array(
46         "marque" => "Peugeot",
47         "couleur" => "rouge",
48         "immatriculation" => "123CD45"
49     ),
50     array(
51         "marque" => "BMW",
52         "couleur" => "noir",
53         "immatriculation" => "789EF67"
54     )
55 );
56
57 // Vérification du contenu du tableau
58 var_dump($voitures);
59
60
61 // 4. Affichage de la liste
62 if (empty($voitures)) {
63     echo "<p>Il n'y a aucune voiture.</p>";
64 } else {
65     echo "<h2>Liste des voitures :</h2>";
66     echo "<ul>";
67
68     foreach ($voitures as $voiture) {
69         echo "<li>Voiture " . $voiture["immatriculation"] .
70             " de marque " . $voiture["marque"] .
71             " (couleur " . $voiture["couleur"] . " )</li>";
72     }
73
74     echo "</ul>";
75
76

```

## Résultat obtenue :

Voici le résultat du script PHP :

Voiture 256AB34 de marque Renault (couleur bleu)

```
array(3) { ["marque"]=> string(7) "Peugeot" ["couleur"]=> string(5) "rouge" ["immatriculation"]=> string(7) "123CD45" }
```

Voiture 123CD45 de marque Peugeot (couleur rouge)

```
array(3) { [0]=> array(3) { ["marque"]=> string(7) "Renault" ["couleur"]=> string(4) "bleu" ["immatriculation"]=> string(7) "256AB34" } [1]=> ["marque"]=> string(3) "BMW" ["couleur"]=> string(4) "noir" ["immatriculation"]=> string(7) "789EF67" }
```

Liste des voitures :

- Voiture 256AB34 de marque Renault (couleur bleu)
- Voiture 123CD45 de marque Peugeot (couleur rouge)
- Voiture 789EF67 de marque BMW (couleur noir)

## Enregistrement du fichier avec git

```
C:\xampp\htdocs\MVC PHP TD1\public_html>git add echo.php

C:\xampp\htdocs\MVC PHP TD1\public_html>git commit -m "TD1 exo4 affichage des variables"
[master 62708f8] TD1 exo4 affichage des variables
1 file changed, 71 insertions(+), 11 deletions(-)

C:\xampp\htdocs\MVC PHP TD1\public_html>git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    page1.html

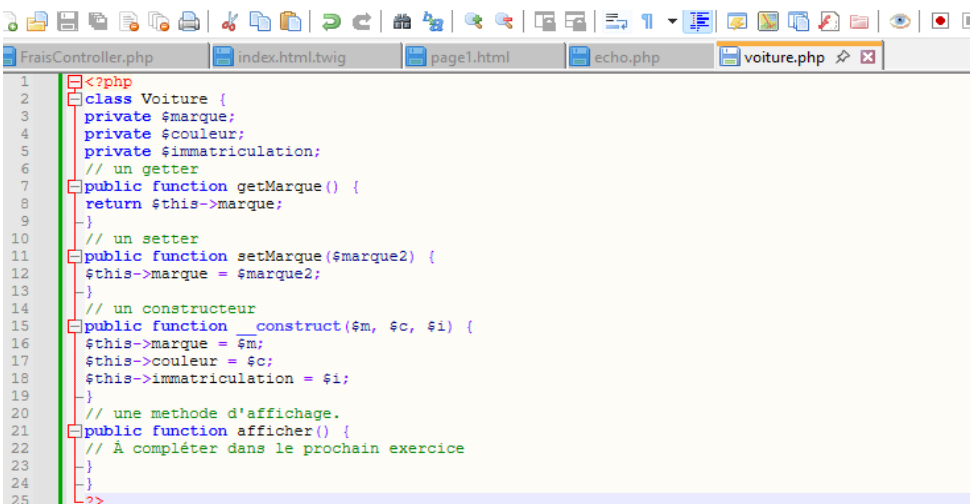
nothing added to commit but untracked files present (use "git add" to track)

C:\xampp\htdocs\MVC PHP TD1\public_html>
```

## Exercice:5

Dans cet exercice, j'ai créé une classe Voiture avec trois attributs : la marque, la couleur et l'immatriculation. J'ai ajouté des getters et des setters pour accéder et modifier ces attributs, ainsi qu'un constructeur pour initialiser une voiture. Le setter de l'immatriculation vérifie qu'elle ne dépasse pas 8 caractères. Enfin, la méthode afficher() permet d'afficher les informations de la voiture.

### Création du fichier voiture.php



```
1 <?php
2 class Voiture {
3     private $marque;
4     private $couleur;
5     private $immatriculation;
6     // un getter
7     public function getMarque() {
8         return $this->marque;
9     }
10    // un setter
11    public function setMarque($marque2) {
12        $this->marque = $marque2;
13    }
14    // un constructeur
15    public function __construct($m, $c, $i) {
16        $this->marque = $m;
17        $this->couleur = $c;
18        $this->immatriculation = $i;
19    }
20    // une methode d'affichage.
21    public function afficher() {
22        // À compléter dans le prochain exercice
23    }
24 }
25 ?>
```

## Enregistrement git

```
C:\xampp\htdocs\MVC PHP TD1\public_html>git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        page1.html
        voiture.php

nothing added to commit but untracked files present (use "git add" to track)

C:\xampp\htdocs\MVC PHP TD1\public_html>git add voiture.php

C:\xampp\htdocs\MVC PHP TD1\public_html>git commit -m"TD1 Exo5 classe voiture"
[master 4647d6f] TD1 Exo5 classe voiture
1 file changed, 64 insertions(+)
create mode 100644 voiture.php

C:\xampp\htdocs\MVC PHP TD1\public_html>git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        page1.html

nothing added to commit but untracked files present (use "git add" to track)

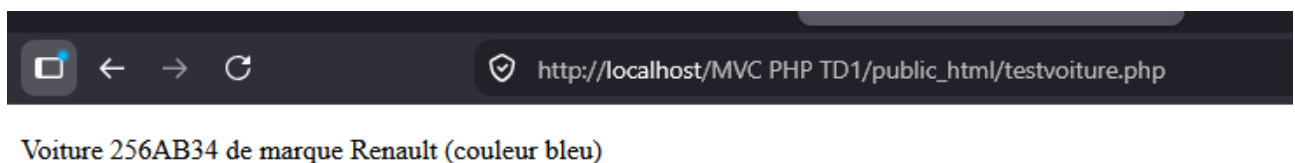
C:\xampp\htdocs\MVC PHP TD1\public_html>
```

La page PHP n'affiche pas d'erreur et elle a bien été enregistrer.

## Exercice : 6

Dans cet exercice, j'ai créé le fichier testVoiture.php et inclus la classe Voiture avec require\_once. J'ai ensuite créé un objet \$voiture1 avec une marque, une couleur et une immatriculation, puis j'ai utilisé la méthode afficher() pour afficher ses informations dans la page web.

Test :



## Exercice:7

Dans cet exercice, j'ai créé un formulaire permettant de saisir l'immatriculation, la marque et la couleur d'une voiture. Le formulaire utilise la méthode GET et envoie les informations vers creerVoiture.php. Lorsque je clique sur « Envoyer », les informations saisies sont transmises dans l'URL sous la forme d'une query string.

```

1 <!DOCTYPE html>
2 <html>
3 <head>
4   <meta charset="utf-8" />
5   <title>Formulaire voiture</title>
6 </head>
7 <body>
8
9   <form method="get" action="creerVoiture.php">
10
11     <fieldset>
12       <legend>Mon formulaire :</legend>
13
14       <p>
15         <label for="immat_id">Immatriculation</label> :
16         <input type="text"
17           placeholder="Ex : 256AB34"
18           name="immatriculation"
19           id="immat_id"
20           required/>
21       </p>
22
23       <p>
24         <label for="marque_id">Marque</label> :
25         <input type="text"
26           placeholder="Ex : Renault"
27           name="marque"
28           id="marque_id"
29           required/>
30       </p>
31
32       <p>
33         <label for="couleur_id">Couleur</label> :
34         <input type="text"
35           placeholder="Ex : bleu"
36           name="couleur"
37           id="couleur_id"
38           required/>
39       </p>
40
41       <p>
42         <input type="submit" value="Envoyer" />
43       </p>
44     </fieldset>
45
46   </form>
47 </body>
48 </html>

```

file:///C:/xampp/htdocs/MVC PHP TD1/public\_html/formulairevoiture.html

---

Mon formulaire :

Immatriculation :

Marque :

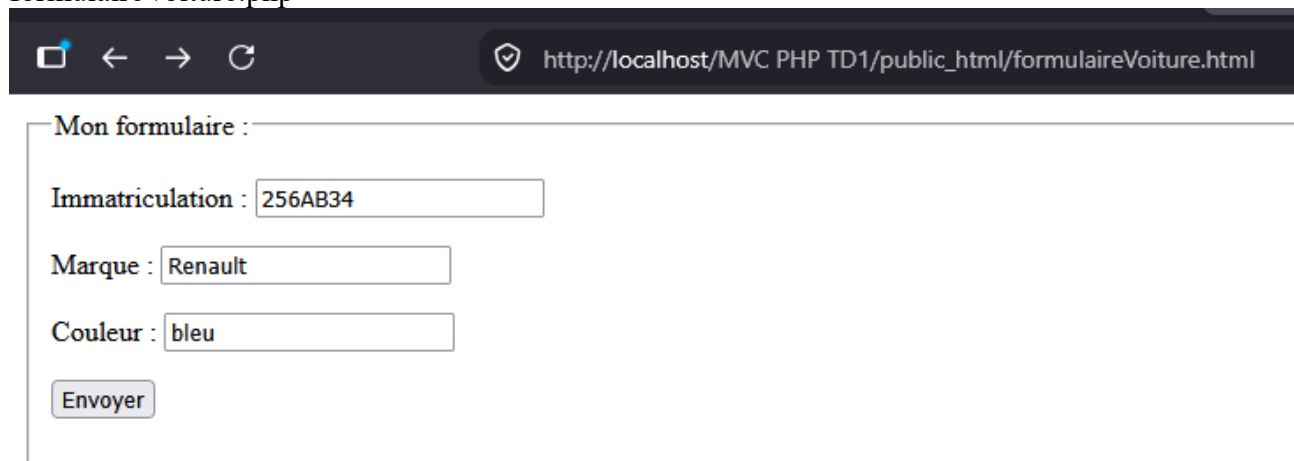
Couleur :

## **Exercice : 8**

Dans cet exercice, j'ai créé le fichier creerVoiture.php afin de récupérer les informations saisies dans le formulaire. J'ai d'abord utilisé la méthode GET, ce qui faisait apparaître les informations dans l'URL. J'ai ensuite modifié le formulaire pour utiliser la méthode POST afin que les informations ne soient plus visibles dans l'URL. J'ai récupéré les données avec le tableau \$\_POST, puis je les ai affichées avec PHP. Enfin, j'ai vérifié la transmission des données avec les outils de développement du navigateur, dans l'onglet Réseau.

La page creerVoiture.php etant créer le formulaire peut fonctionner voici un test :

formulaireVoiture.php



Mon formulaire :

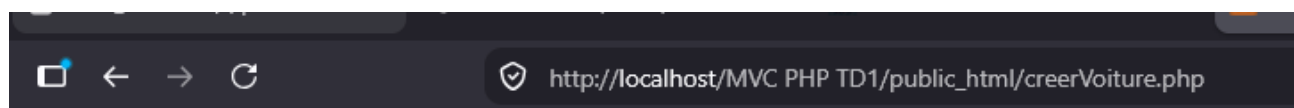
Immatriculation : 256AB34

Marque : Renault

Couleur : bleu

Envoyer

creerVoiture.php



Voiture 256AB34 de marque Renault (couleur bleu)

## **Exercice : 9**

Dans cet exercice, j'ai créé les classes Utilisateur et Trajet pour représenter les éléments d'un site de covoiturage. J'ai créé les différents attributs de chaque classe et utilisé des getters et setters génériques pour pouvoir récupérer et modifier leurs valeurs. J'ai également créé un constructeur qui prend un tableau en paramètre afin d'initialiser facilement les attributs de chaque objet.

### Utilisateur.php

```
<?php
class Utilisateur
{
    private $login;
    private $nom;
    private $prenom;

    public function get($nom_attribut)
    {
        return $this->$nom_attribut;
    }

    public function set($nom_attribut, $valeur)
    {
        $this->$nom_attribut = $valeur;
    }

    public function __construct($data)
    {
        foreach ($data as $attribut => $valeur) {
            $this->$attribut = $valeur;
        }
    }
}
```

### Trajet.php

```
<?php
class Trajet
{
    private $id;
    private $depart;
    private $arrivee;
    private $date;
    private $nbplaces;
    private $prix;
    private $conducteur_login;

    public function get($nom_attribut)
    {
        return $this->$nom_attribut;
    }

    public function set($nom_attribut, $valeur)
    {
        $this->$nom_attribut = $valeur;
    }

    public function __construct($data)
    {
        foreach ($data as $attribut => $valeur) {
            $this->$attribut = $valeur;
        }
    }
}
```

## **Conclusion**

Ce TP nous a permis de revoir le fonctionnement d'une page web et la différence entre l'ouverture directe d'un fichier et son accès via un serveur HTTP. Nous avons également commencé à utiliser PHP, qui permettra par la suite de créer des pages web dynamiques.